JavaScript is disabled on your browser.

Java SE 27 & JDK 27

Interface SeekableByteChannel

All Superinterfaces:

AutoCloseable, ByteChannel, Channel, Closeable, ReadableByteChannel, WritableByteChannel

All Known Implementing Classes:

FileChannel

```
public interface SeekableByteChannel
extends ByteChannel
```

A byte channel that maintains a current *position* and allows the position to be changed.

A seekable byte channel is connected to an entity, typically a file, that contains a variable-length sequence of bytes that can be read and written. The current position can be *queried* and *modified*. The channel also provides access to the current *size* of the entity to which the channel is connected. The size increases when bytes are written beyond its current size; the size decreases when it is *truncated*.

The position and truncate methods which do not otherwise have a value to return are specified to return the channel upon which they are invoked. This allows method invocations to be chained. Implementations of this interface should specialize the return type so that method invocations on the implementation class can be chained.

Since:

1.7

See Also:

Files.newByteChannel(Path, Set, FileAttribute...)

Method Summary

All Methods | Instance Methods

Modifier and Type	Method	Description
long	**position**()	Returns this channel's position.
SeekableByteChannel	**position**(long newPosition)	Sets this channel's position.
int	**read**(ByteBuffer dst)	

		Reads a sequence of bytes from this channel into the given buffer.
long	**size**()	Returns the current size of entity to which this channel is connected.
SeekableByteChannel	**truncate**(long size)	Truncates the entity, to which this channel is connected, to the given size.
int	**write**(ByteBuffer src)	Writes a sequence of bytes to this channel from the given buffer.

Methods declared in interface Channel

close, isOpen

Method Details

read

```
int read(ByteBuffer dst)
  throws IOException
```

Reads a sequence of bytes from this channel into the given buffer.

Bytes are read starting at this channel's current position, and then the position is updated with the number of bytes actually read. Otherwise this method behaves exactly as specified in the ReadableByteChannel interface.

Specified by:

read in interface ReadableByteChannel

Parameters:

dst - The buffer into which bytes are to be transferred

Returns:

The number of bytes read, possibly zero, or -1 if the channel has reached end-of-stream

Throws:

ClosedChannelException - If this channel is closed

AsynchronousCloseException - If another thread closes this channel while the read operation is in progress

ClosedByInterruptException - If another thread interrupts the current thread while the read operation is in progress, thereby closing the channel and setting the current thread's interrupted status

NonReadableChannelException - If this channel was not opened for reading

IOException - If some other I/O error occurs

write

```
int write(ByteBuffer src)
   throws IOException
```

Writes a sequence of bytes to this channel from the given buffer.

Bytes are written starting at this channel's current position, unless the channel is connected to an entity such as a file that is opened with the APPEND option, in which case the position is first advanced to the end. The entity to which the channel is connected is grown, if necessary, to accommodate the written bytes, and then the position is updated with the number of bytes actually written. Otherwise this method behaves exactly as specified by the WritableByteChannel interface.

Specified by:

write in interface WritableByteChannel

Parameters:

src - The buffer from which bytes are to be retrieved

Returns:

The number of bytes written, possibly zero

Throws:

ClosedChannelException - If this channel is closed

AsynchronousCloseException - If another thread closes this channel while the write operation is in progress

ClosedByInterruptException - If another thread interrupts the current thread while the write operation is in progress, thereby closing the channel and setting the current thread's interrupted status

NonWritableChannelException - If this channel was not opened for writing

IOException - If some other I/O error occurs

position

```
long position()
       throws IOException
```

Returns this channel's position.

Returns:

This channel's position, a non-negative integer counting the number of bytes from the beginning of the entity to the current position

Throws:

`ClosedChannelException` - If this channel is closed

`IOException` - If some other I/O error occurs

position

```
SeekableByteChannel position(long newPosition)
                     throws IOException
```

Sets this channel's position.

Setting the position to a value that is greater than the current size is legal but does not change the size of the entity. A later attempt to read bytes at such a position will immediately return an end-of-file indication. A later attempt to write bytes at such a position will cause the entity to grow to accommodate the new bytes; the values of any bytes between the previous end-of-file and the newly-written bytes are unspecified.

Setting the channel's position is not recommended when connected to an entity, typically a file, that is opened with the `APPEND` option. When opened for append, the position is first advanced to the end before writing.

Parameters:

`newPosition` - The new position, a non-negative integer counting the number of bytes from the beginning of the entity

Returns:

This channel

Throws:

`ClosedChannelException` - If this channel is closed

`IllegalArgumentException` - If the new position is negative

`IOException` - If some other I/O error occurs

size

```
long size()
   throws IOException
```

Returns the current size of entity to which this channel is connected.

Returns:

The current size, measured in bytes

Throws:

`ClosedChannelException` - If this channel is closed

`IOException` - If some other I/O error occurs

truncate

```
SeekableByteChannel truncate(long size)
                     throws IOException
```

Truncates the entity, to which this channel is connected, to the given size.

If the given size is less than the current size then the entity is truncated, discarding any bytes beyond the new end. If the given size is greater than or equal to the current size then the entity is not modified. In either case, if the current position is greater than the given size then it is set to that size.

An implementation of this interface may prohibit truncation when connected to an entity, typically a file, opened with the `APPEND` option.

Parameters:

`size` - The new size, a non-negative byte count

Returns:

This channel

Throws:

`NonWritableChannelException` - If this channel was not opened for writing

`ClosedChannelException` - If this channel is closed

`IllegalArgumentException` - If the new size is negative

`IOException` - If some other I/O error occurs

Report a bug or suggest an enhancement
For further API reference and developer documentation see the Java SE Documentation, which contains more detailed, developer-targeted descriptions with conceptual overviews, definitions of terms, workarounds, and working code examples. Other versions.
Java is a trademark or registered trademark of Oracle and/or its affiliates in the US and other countries.

Copyright © 1993, 2026, Oracle and/or its affiliates, 500 Oracle Parkway, Redwood Shores, CA 94065 USA.
All rights reserved. Use is subject to license terms and the documentation redistribution policy.

Scripting on this page tracks web page traffic, but does not change the content in any way.